國立臺灣大學電機資訊學院資訊工程學系

碩士論文

Department of Computer Science and Information Engineering

College of Electrical Engineering and Computer Science

National Taiwan University

Master's Thesis

大語言模型偵測價格操縱攻擊

Detecting Price Manipulation Attacks Using Large Language Models

葉富銘

Ye, Fu-Ming

指導教授: 蕭旭君 博士

Advisor: Hsu-Chun Hsiao, Ph.D.

中華民國 115 年 7 月

July 2026

doi:10.5566/NTU2026XXXXX

國立臺灣大學碩士學位論文

口試委員會審定書

大語言模型偵測價格操縱攻擊

Detecting Price Manipulation Attacks Using Large Language Models

本論文係葉富銘君（R13922146）在國立臺灣大學資訊工程學系完成之碩士學位論文，於民國 115 年 7 月 18 日承下列考試委員審查通過及口試及格，特此證明

口試委員：_____

（指導教授）

_____ _____

_____ _____

_____ _____

_____ _____

所　　長：_____

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

Acknowledgements

Thanks

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

摘要

隨著去中心化金融快速發展，智慧合約承載的資產規模與金融邏輯日益複雜，其安全性也成為區塊鏈應用能否被信任的關鍵。價格操縱漏洞，尤其在可由閃電貸放大影響的情境中，並不只是單一程式語法錯誤；其風險取決於合約如何取得價格、該價格如何影響清算、抵押品估值等關鍵金融操作，以及系統是否具備足夠的保護機制。

過往已有許多研究嘗試以交易分析、靜態分析或大型語言模型輔助智慧合約漏洞偵測。然而，既有工作多以攻擊事件、單一合約、整體協定標記或二元分類結果作為主要評估單位，與真實審計流程仍有落差。審計人員通常面對的是完整的協定程式碼庫，並需要能指出可疑程式位置、根因、影響、修補方向與支持證據的發現項目。

本研究以協助審計人員快速定位閃電貸價格操縱漏洞為目標，設計並實作一套結合靜態分析與大型語言模型推論的分析流程。系統先從完整程式碼庫中萃取候選資料流證據與相關程式脈絡，再透過路徑摘要與分組降低重複資訊，最後由分階段大型語言模型流程產生結構化的審計式發現項目。每一項發現皆包含可疑程式位置、根因說明、影響描述、缺失的保護機制與支持證據，使審計人員能回溯工具判斷並進一步驗證。評估方面，本研究結合協定層級的精確率、召回率與調和平均分數，以及以公開審計報告發現項目為對照的報告層級評估。

doi:10.5566/NTU2026XXXXX

研究結果顯示，此流程能將大量原始資料流證據收斂為較少且可追溯的審計式發現項目，並揭露二元分類指標難以呈現的失敗模式，例如錯誤的程式定位、過多不受支持的報告，以及根因解釋偏移。本研究的主要貢獻在於將大型語言模型輔助智慧合約漏洞偵測的評估重心，從是否偵測到漏洞推進到是否能幫助審計人員在完整程式碼庫中快速定位並理解閃電貸價格操縱漏洞。

關鍵字：去中心化金融、智慧合約安全、價格操縱、閃電貸、大型語言模型、靜態污點分析、審計輔助

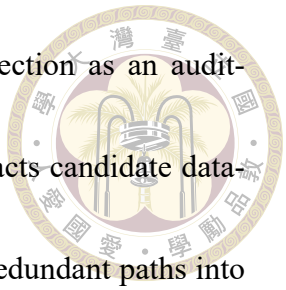
doi:10.5566/NTU2026XXXXX

Abstract

Decentralized finance (DeFi) has made smart contracts responsible for increasingly complex financial logic and high-value assets. As a result, smart contract security has become central to the reliability of blockchain applications. Flash-loan price manipulation is a particularly challenging vulnerability class because the risk cannot be determined from syntax alone. Whether a vulnerability exists depends on how a contract obtains price information, how that value affects liquidation, collateral valuation, or other financially sensitive operations, and whether sufficient protection is applied.

Prior research has explored transaction analysis, static analysis, and large language models (LLMs) for smart contract vulnerability detection. However, many evaluations still focus on attack events, individual contracts, protocol-level labels, or binary classification results. This does not fully match real audit workflows, where auditors inspect an entire protocol repository and expect findings that identify suspicious code locations, root causes, impacts, mitigation strategies, and supporting evidence.

doi:10.5566/NTU2026XXXXX



This thesis therefore studies flash-loan price manipulation detection as an audit-assistance problem. It designs and implements a workflow that extracts candidate data-flow evidence and relevant code context from a repository, reduces redundant paths into representative evidence, and uses a staged LLM workflow to generate structured audit-style findings. Each finding reports the suspicious code location, root cause, impact, missing protection, and supporting evidence so that auditors can trace and verify the tool's judgment. The evaluation combines protocol-level detection metrics with report-level comparison against public audit findings.

The results show that the workflow can reduce large sets of raw data-flow evidence into fewer traceable audit-style findings while exposing failure modes that binary metrics tend to hide, including incorrect code localization, unsupported extra reports, and root-cause drift. The main contribution of this thesis is to shift the evaluation of LLM-assisted smart contract security tools toward whether they help auditors quickly localize and understand flash-loan price manipulation vulnerabilities in complete repositories.

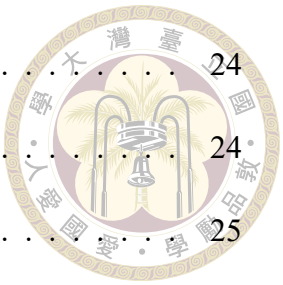
Keywords: Decentralized Finance, Smart Contract Security, Price Manipulation, Flash Loans, Large Language Models, Static Taint Analysis, Audit Assistance

doi:10.5566/NTU2026XXXXX

Contents

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX



doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

List of Figures

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

List of Tables

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

Denotation

AMM	Automated Market Maker.
API	Application Programming Interface.
CFG	Control-Flow Graph.
CSV	Comma-Separated Values.
DeFi	Decentralized Finance.
EVM	Ethereum Virtual Machine.
F1-score	The harmonic mean of precision and recall.
IR	Intermediate Representation.
JSON	JavaScript Object Notation.
LLM	Large Language Model.
PMDetector	A prior hybrid framework for detecting price manipulation vulnerabilities in DeFi protocols.

doi:10.5566/NTU2026XXXXX

RQ	Research Question.
SSA	Static Single Assignment.
Slither	A static analysis framework for Solidity smart contracts.
SlithIR	The intermediate representation exposed by Slither.
TWAP	Time-Weighted Average Price.
p	A project in the over-reporting penalty, or a predicted explanation in the semantic explanation score, depending on context.
g	A ground-truth audit-finding explanation in the semantic explanation score.
$E(\cdot)$	The embedding function used to map report text into a vector representation.
τ	The semantic similarity threshold used when computing the description score.
$\cos(E(p), E(g))$	Cosine similarity between the predicted explanation embedding and the ground-truth explanation embedding.
DescriptionScore(p, g)	The semantic explanation score between a predicted report and a ground-truth finding.
Penalty(p)	The over-reporting penalty for project p.
#Predictions(p)	The number of generated findings for project p.

doi:10.5566/NTU2026XXXXX

#GroundTruth(p)	The number of relevant ground-truth audit findings for project p.

doi:10.5566/NTU2026XXXXX

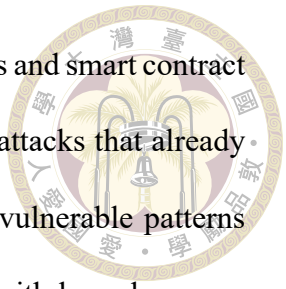
doi:10.5566/NTU2026XXXXX

Chapter 1 Introduction

Decentralized finance (DeFi) protocols encode financial logic directly in smart contracts. A lending protocol determines whether a position is liquidatable from collateral prices, an automated market maker updates liquidity and swap states from pool values, and a vault accounts for deposits, debt, and rewards from asset balances and reserve data. When these values are read from manipulable spot prices, stale or misconfigured oracles, or insufficiently protected price helper functions, attackers can temporarily distort the protocol's view of value and induce incorrect economic behavior. As a result, price manipulation has become an important and difficult vulnerability class in smart contract security.

Flash-loan price manipulation is especially difficult to reason about because the vulnerability is rarely a simple syntactic bug. The same price read may be safe or unsafe depending on how it is computed, how fresh it is, what protection surrounds it, and which financial decision later depends on it. For example, using a current pool price directly in collateral valuation or liquidation logic can be dangerous, while a properly configured time-weighted average price (TWAP), independent oracle check, deviation bound, or slippage control may make a similar-looking code pattern acceptable. The problem is therefore contextual: detectors that only match fixed code patterns often miss the economic conditions that determine exploitability.

doi:10.5566/NTU2026XXXXX

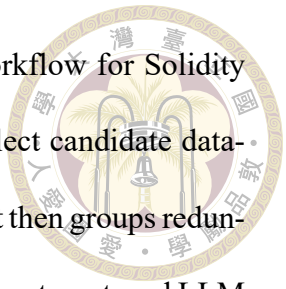


Prior work has made substantial progress in detecting DeFi attacks and smart contract vulnerabilities. Some systems analyze historical transactions to find attacks that already occurred. Others inspect smart contract code or bytecode to detect vulnerable patterns before exploitation. Recent approaches also combine static analysis with large language models (LLMs) to reason about code context and economic intent. Recent audit corpora and benchmarks have also begun to structure repository-level evaluation for smart contract agents. These directions are useful, but they do not yet make repository-level flash-loan price manipulation findings for auditors the primary evaluation target.

Real audits have a different shape. Auditors typically inspect an entire protocol repository rather than a single function or isolated transaction. They must understand how contracts, helper libraries, price adapters, accounting logic, and external integrations interact. The final audit report is also not a binary label. A useful finding identifies the affected code location or snippet, explains the root cause, describes the impact, recommends mitigation, and provides enough evidence for the auditor and developer to verify the issue. A tool that produces only a protocol-level label, or too many weakly supported findings, still leaves substantial audit work to humans.

This thesis therefore studies flash-loan price manipulation detection as an audit-assistance problem. The goal is not to replace human auditors, but to help them quickly locate and understand suspicious price-dependent logic in a complete repository. A useful assistant should gather concrete code evidence, reduce redundant paths, explain why a particular price dependency is risky, and produce findings whose structure resembles the information auditors already use: code location, root cause, impact, missing protection, and supporting evidence.

doi:10.5566/NTU2026XXXXX



To study this problem, this thesis designs and implements a workflow for Solidity projects. The workflow uses Slither-based static analysis [12] to collect candidate data-flow evidence related to price dependencies and financial operations. It then groups redundant evidence into representative entries and passes the reduced evidence to a staged LLM workflow. The LLM stages are constrained to reason over the extracted evidence and to emit structured findings rather than unconstrained prose. Each final finding is designed to be traceable back to static-analysis artifacts, so that auditors can inspect the evidence rather than trust a free-form model judgment.

This thesis also proposes an evaluation perspective aligned with audit workflows. Protocol-level precision, recall, and F1-score are still useful as baseline measurements, but they do not show whether a generated report points to the right code, explains the right root cause, recommends the right mitigation, or avoids excessive false positives. To evaluate audit usefulness, this thesis uses public audit findings from sources such as Solodit, Code4rena, and Sherlock as report-level comparison targets [2, 3, 10]. It measures localization, semantic explanation similarity, impact and mitigation agreement, and over-reporting penalties that approximate manual review burden.

In particular, this thesis is organized around three research questions. First, can a static-analysis-plus-LLM workflow generate audit-style findings that help auditors locate flash-loan price manipulation vulnerabilities in a repository? Second, what failure modes are hidden when evaluation stops at binary vulnerability classification? Third, can report-level evaluation based on audit findings better reflect audit usefulness than protocol-level precision and recall alone?

The contributions of this thesis are as follows:

doi:10.5566/NTU2026XXXXX

1. We frame flash-loan price manipulation detection as an audit-assistance problem, emphasizing repository-level localization, explanation quality, and false-positive burden rather than only vulnerable or benign classification.

2. We present a workflow that combines static evidence extraction, path reduction, staged LLM reasoning, and structured audit-style finding generation for Solidity repositories.

3. We propose an audit-finding-based evaluation methodology that compares generated reports with public audit findings using localization, root-cause explanation, impact and mitigation alignment, semantic similarity, and over-reporting metrics.

The remainder of this thesis is organized as follows. Chapter 2 provides background and reviews related work. Chapters 3 and 4 present the design and implementation of the proposed workflow. Chapter 5 presents the evaluation framework and qualitative results. Chapter 6 concludes the thesis and discusses limitations and future work.

doi:10.5566/NTU2026XXXXX

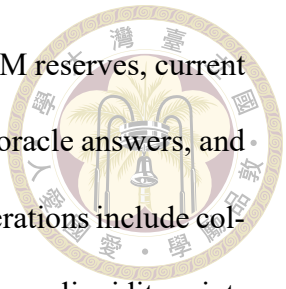
Chapter 2 Background and Related Work

2.1 Price Manipulation in DeFi

Price manipulation vulnerabilities arise when a protocol relies on a price-related value that can be influenced by an attacker. In DeFi systems, price is not only used for swaps. It also affects collateral valuation, liquidation eligibility, minting and burning, vault share calculation, reward accounting, margin requirements, and other economically sensitive state transitions. A short-lived price deviation may therefore be enough to extract value if the protocol reads the manipulated value and immediately commits an irreversible state change.

Many such attacks are amplified by flash loans or other low-cost temporary liquidity movements. The attacker first distorts the apparent market state, then triggers a price-dependent operation, and finally restores the pool state before the transaction ends. This makes the vulnerable condition highly context-dependent. Detecting the issue requires understanding how a price is obtained, how it is propagated through the program, and where it influences a financial decision. Merely observing that a contract reads an oracle interface or a pool state variable is not sufficient.

doi:10.5566/NTU2026XXXXX



Common risky price dependencies include direct reads from AMM reserves, current pool ticks, square-root prices, direct spot-price helper functions, stale oracle answers, and protocol-specific wrappers around those values. Common affected operations include collateral checks, liquidation logic, margin updates, swap output calculations, liquidity minting, reward release, and accounting state writes. A safe implementation often adds protection such as time-weighted average prices, independent oracle cross-checks, staleness checks, deviation bounds, slippage checks, or delayed settlement. The central difficulty of price manipulation is therefore not recognizing one dangerous API call, but evaluating the relationship among price source, economic sink, and missing protection in the broader protocol context.

2.2 Smart Contract Auditing Practice

Smart contract auditing is typically conducted at the level of a complete protocol repository rather than an isolated function or transaction. Auditors inspect multiple contracts, helper libraries, inherited modules, deployment assumptions, integration points, tests, and business logic together. For DeFi protocols, this often means tracing how a price is computed, how it crosses contract boundaries, and where it influences economically sensitive state transitions.

The output of an audit is also richer than a vulnerability label. A typical finding contains a title, severity, affected code location or snippet, root cause, impact, supporting evidence or proof-of-concept context, and mitigation recommendation. These elements matter because developers must reproduce and fix the issue, while auditors must justify why the reported behavior is risky. An automated tool that aims to assist audits should

doi:10.5566/NTU2026XXXXX

therefore be judged not only by whether it predicts that a protocol is vulnerable, but by whether it produces findings that auditors can directly inspect and validate.

2.3 Related Work

2.3.1 Price Manipulation Detection Systems

One major direction studies price manipulation through transaction-level or post-hoc analysis. DeFiRanger reconstructs high-level DeFi semantics from raw Ethereum transactions by building cash-flow trees and lifting low-level token transfers into operations such as trades and liquidity changes [16]. DeFiScope extends this line by combining transaction-level DeFi operations with LLM-based abstraction of price calculation logic from source code, and targets both standard and custom price models [19]. These systems show that price manipulation cannot be understood from syntax alone and instead requires reasoning about financial behavior and price semantics. However, their primary output unit remains attack or transaction detection, which is valuable for monitoring and incident analysis but does not directly answer whether an auditor can obtain a precise pre-deployment finding from a repository.

Another direction targets code-based or pre-mortem detection. Static analysis is attractive in this setting because smart contract code is transparent and many vulnerability patterns leave structural traces in control flow, data flow, and cross-contract interactions. Slither provides a widely used foundation for such analyses by parsing Solidity code, building intermediate representations, and exposing program information suitable for custom security analysis [12]. Taint analysis is especially relevant for price manipulation

doi:10.5566/NTU2026XXXXX

because it can model whether a price-like value reaches a financially sensitive operation. In this thesis, source and sink are analysis abstractions rather than vulnerability labels. They identify potentially relevant flows, while exploitability still depends on economic meaning and missing protection.

Recent pre-mortem systems move closer to this thesis. PriceSleuth identifies core price-calculation logic, uses an LLM to locate price-calculation statements, and then applies dependency and propagation analysis to detect state-manipulation vulnerabilities [15]. PMDetector combines formal attack modeling, static taint analysis, LLM-based filtering and attack simulation, and static validation to detect DeFi price manipulation vulnerabilities [5]. Both works are closer to audit assistance than pure post-hoc attack detection because they reason about code before exploitation. Nevertheless, their reported evaluation is still centered mainly on detection performance over vulnerable and benign contracts or protocols.

Some recent systems also begin to frame the problem as an adaptive auditing workflow. SmartAuditFlow introduces a plan-execute framework in which the LLM dynamically revises its audit strategy based on intermediate outputs, integrates external knowledge such as static analysis and retrieval-augmented generation, and produces comprehensive security reports [13]. This direction is important because it moves the field closer to multi-stage audit reasoning rather than one-shot classification. Still, the target remains general smart contract auditing, and the evaluation does not focus on repository-level flash-loan price manipulation findings with explicit source-sink-mitigation alignment.

doi:10.5566/NTU2026XXXXX

2.3.2 LLM-Assisted Detection

Beyond price manipulation, a broader line of work studies how LLMs can assist smart contract vulnerability detection. GPTScan is an early and influential example that combines GPT with static analysis for smart contract logic vulnerability detection [11]. Instead of treating the LLM as a standalone oracle, GPTScan uses it as a code-understanding component to match candidate vulnerabilities against scenario descriptions and then validates key variables and statements through static confirmation. This hybrid design is important because it explicitly addresses one of the main weaknesses of direct LLM-based detection: high false-positive rates.

Other work focuses on improving reasoning structure within the LLM itself. GPTLens frames smart contract vulnerability detection as a two-stage process in which one LLM role generates candidate vulnerabilities and another role discriminates among them [4]. The paper argues that there is a tension between maximizing recall and controlling false positives, and that multi-stage reasoning can partially manage this trade-off. This perspective is relevant to audit assistance because an auditor needs both broad coverage and low noise; simply sampling more candidate answers is not enough.

A more model-centric direction improves the smart-contract-specific capability of LLMs through domain adaptation and explanation-oriented training. Smart-LLaMA introduces smart-contract-specific continual pre-training and explanation-guided fine-tuning, and explicitly targets vulnerability detection, explanation quality, and precise vulnerability location [17]. This is particularly relevant to the present thesis because it shows that LLM-based systems can be optimized not only for binary detection, but also for localization and explanation, which are closer to the needs of auditing.

doi:10.5566/NTU2026XXXXX

Another related direction treats auditing as a collaborative multi-agent process. LLM-SmartAudit uses a multi-agent conversational framework to detect and analyze smart contract vulnerabilities, and reports stronger performance than traditional auditing tools on its evaluation datasets [14]. While the vulnerability scope is broader than price manipulation, the paper reflects a growing trend: LLM systems are increasingly designed not merely to classify contracts, but to simulate aspects of audit reasoning and report generation.

PMDetector is best understood as a price-manipulation-specific extension of this broader LLM-assisted literature. It inherits the general move toward hybrid reasoning, staged analysis, and richer semantic interpretation, but applies those ideas to a narrow and economically important vulnerability class [5]. This thesis builds on the same general intuition, yet shifts the evaluation target further toward whether the final output is useful as a repository-level audit finding.

2.3.3 Audit Report Corpora and Benchmarks

Public audit reports and contest findings provide a natural source of real-world security artifacts. Platforms such as Solodit aggregate findings from multiple audit firms and contests, while Code4rena and Sherlock publish contest reports and discussions that contain vulnerability descriptions, affected code regions, impact statements, and remediation guidance [2, 3, 10]. These artifacts are valuable because they more closely resemble the output expected from an audit assistant than coarse exploit labels or contract-level vulnerability tags.

Recent work has begun to transform such reports into structured corpora. FORGE uses an LLM-driven pipeline to extract self-contained vulnerability information from real-

doi:10.5566/NTU2026XXXXX

world audit reports and classify the resulting findings under the CWE hierarchy [1]. GiAnt likewise distills real-world audit reports into structured findings and then applies quality assurance with an LLM-as-a-judge mechanism to produce a large audit-oriented corpus [18]. These works demonstrate that audit findings can be normalized into machine-usable records with fields such as vulnerability description, severity, location, impact, and mitigation, thereby making report-level evaluation more feasible.

In parallel, benchmark design has also shifted toward repository-level audit tasks. ScaBench provides curated datasets from recent real-world smart contract audits together with official scoring tools for evaluating AI audit agents [9]. EVMbench further expands the benchmark space by evaluating AI agents in detect, patch, and exploit modes over curated vulnerabilities drawn from historical audits [8]. Hound, in turn, uses a subset of ScaBench to study system-level reasoning in security audits through relation-first knowledge graphs and hypothesis-driven analysis [6]. These benchmarks move evaluation closer to practical auditing by treating repositories, vulnerability reports, and agent behavior as first-class benchmark objects rather than reducing everything to file-level labels.

Taken together, the literature reviewed in this chapter establishes the main ingredients needed by this thesis: price-manipulation reasoning, hybrid static-analysis-plus-LLM workflows, and repository-level audit artifacts and benchmarks. A clear gap nevertheless remains. Existing systems and benchmarks are still largely general-purpose: they do not directly provide an evaluation framework tailored to flash-loan price manipulation findings in which source identification, sink localization, mitigation agreement, semantic explanation quality, and over-reporting cost are measured together. This thesis therefore draws methodological inspiration from these directions while focusing on a narrower,

doi:10.5566/NTU2026XXXXX

audit-oriented evaluation target.

doi:10.5566/NTU2026XXXXX

Chapter 3 Design

3.1 Design Goals

The goal of this thesis is to design a price manipulation analysis workflow whose output can be evaluated as an audit report, not only as a binary vulnerability label. This leads to four design goals.

Evidence first. The system should not ask an LLM to find vulnerabilities from an entire repository without structure. Static analysis must first provide concrete source, sink, and path evidence.

Semantic grouping. Raw taint paths are often redundant. A single root cause may produce many syntactic paths because of SSA variables, helper functions, tuple returns, or multiple sink operands. The system should group paths into representative semantic units before LLM analysis.

Structured reasoning. LLM output should be constrained to explicit fields such as source classification, sink impact decision, root-cause location, missing mitigation, and supporting evidence IDs. Free-form prose alone is difficult to validate and score.

Audit-oriented evaluation. The final output should be comparable to audit findings.

doi:10.5566/NTU2026XXXXX

The evaluation should measure localization, explanation match, and over-reporting, not only protocol-level detection.

3.2 Workflow Overview

The proposed workflow contains four major phases:

1. Static taint analysis identifies candidate flows from price-related sources to economically meaningful sinks.

2. Path abstraction canonicalizes sources, records provenance, groups sink operands, and emits representative paths.

3. LLM-based analysis filters retained evidence and generates structured audit-style findings.

4. Report-level evaluation matches predictions against audit findings and computes binary, localization, semantic, and over-reporting metrics.

This design follows the high-level direction of PMDetector, but changes the evaluation target. PMDetector-style taint analysis is used as the front end, and LLM reasoning is used as the semantic filter. The difference is that this thesis treats the generated report as the main artifact.

doi:10.5566/NTU2026XXXXX

3.3 Static Analysis Design

3.3.1 Source Model

The source model intentionally over-approximates possible price-related values. Candidate sources include oracle-like functions, AMM reserve and price reads, external view or pure calls with price-like usage, flash-loan-related calls, and storage reads from price-like state variables. This broad definition is necessary because DeFi protocols often wrap price reads behind protocol-specific helper functions.

Each source is represented not only as a raw variable but also as a canonical source record. A canonical source groups multiple exact seed identifiers that refer to the same semantic source. For example, a helper function may return a price that is unpacked into several local variables across different callers. Treating these as unrelated sources would inflate the number of findings. Canonicalization allows the system to report one root cause with multiple supporting paths.

3.3.2 Sink Model

The sink model focuses on economically meaningful operations. Candidate sinks include price calculations, collateral checks, lending and borrowing operations, margin updates, liquidation logic, token minting and burning, transfers, and state writes that affect accounting. The sink model is broader than the final vulnerability definition because static analysis should first collect possible evidence.

For evaluation, it is important to record the exact sink operand. A function may

doi:10.5566/NTU2026XXXXX

contain several state writes, but only one operand may be controlled by a manipulable price source. The design therefore tracks semantic sink operands instead of reporting only that a function contains a sink.

3.3.3 Propagation and Path Reconstruction

Taint propagation operates over Slither IR. The design propagates taint through assignments, arithmetic operations, references, member accesses, tuple unpacking, return values, internal calls, library calls, and selected inter-contract calls. The analysis also records provenance so that a local wrapper can be connected to the terminal external read.

After propagation, the system reconstructs paths from sink operands back to their sources. The raw path list is expected to contain duplicates and near duplicates. This is acceptable at the static-analysis stage because the goal is to avoid missing potentially relevant flows. Redundancy is reduced in the path abstraction phase.

3.4 Representative Path Design

The representative path layer converts raw static-analysis results into compact LLM evidence. It has three responsibilities.

First, it collapses duplicate raw paths that share the same semantic source-sink relationship. This reduces prompt size and prevents the LLM from mistaking path volume for vulnerability severity.

Second, it preserves information needed for audit reporting: source label, source function, source provenance, terminal external reads, sink function, marked sink lines,

doi:10.5566/NTU2026XXXXX

state writes, external calls, and path counts. The LLM should see enough context to reason about exploitability without receiving the entire repository.

Third, it assigns stable evidence identifiers. The final LLM report refers to supporting evidence IDs, making it possible to trace each generated finding back to concrete static-analysis artifacts.

3.5 LLM Analysis Design

3.5.1 Stage 0: Source Classification

The first LLM stage classifies each canonical source. The allowed classifications are `spot_amm_price`, `twap_or_oracle_price`, `protocol_state`, `fee_or_liquidity`, `unrelated`, and `unknown`. The model also outputs a decision: keep, drop, or need more context.

This stage is designed to remove obvious false positives early. For example, pool addresses, DAO addresses, balances, total supply, and fee-growth helpers may be part of a taint path but are not necessarily manipulable price sources.

3.5.2 Stage 1: Sink Group Triage

For retained sources, the second LLM stage judges whether each sink group is economically dangerous. A sink is dangerous when the tainted value affects accounting, margin, collateral, liquidation, value transfer, minting, burning, or another price-sensitive operation. It is not dangerous when it is telemetry, metadata, harmless caching, or a value not actually controlled by the retained source.

doi:10.5566/NTU2026XXXXX

3.5.3 Stage 2: Root-Cause Judgment

The final LLM stage emits at most one finding per canonical source. This design reduces over-reporting by preventing every path from becoming a separate finding. Each output includes a verdict, root-cause location, vulnerable external read, missing mitigation, supporting evidence IDs, duplicate or supporting evidence IDs, and a concise explanation.

The output schema is intentionally audit-like. It should be possible to compare the generated explanation with a ground-truth audit finding and ask whether they describe the same issue.

3.6 Evaluation Design

3.6.1 Ground Truth

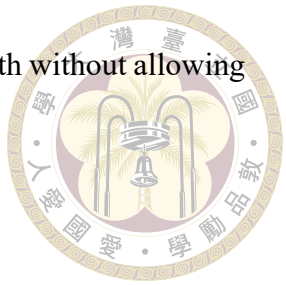
The proposed ground truth is a curated set of audit findings related to price manipulation. Each finding should contain at least the protocol, title, description, affected function or code location when available, impact, and mitigation. The dataset should also record whether source code is available and whether the repository is build-verified.

3.6.2 Report Matching

For each project, generated findings are matched against ground-truth audit findings. A pair score combines localization agreement, mitigation agreement, and semantic explanation similarity. Low-scoring pairs are treated as non-matches. Maximum-weight

doi:10.5566/NTU2026XXXXX

bipartite matching can then be used to assign predictions to ground truth without allowing one prediction to satisfy multiple findings.



3.6.3 Over-Reporting Penalty

The evaluation penalizes unsupported extra findings. For a project p, the basic penalty is:

$$\text{Penalty}(p) = \max(0, \#\text{Predictions}(p) - \#\text{GroundTruth}(p)).$$

This penalty captures audit workload. A detector that emits many weak reports can increase recall, but it also increases manual review cost.

3.7 Scope

The initial scope is price manipulation in Solidity-based DeFi protocols. The dataset may contain related reward or vote manipulation findings, but the core thesis focuses on price-related sources and price-sensitive sinks. Other vulnerability classes such as reentrancy, access control, and bridge-specific logic are outside the main scope unless they directly affect price manipulation mitigation.

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

Chapter 4 Implementation

4.1 System Overview

The system is implemented in Python and built on top of Slither. The main entry point is a price manipulation analyzer that accepts a Solidity file or project directory, initializes Slither, constructs source and sink managers, and runs a project-level taint tracker. The system also includes command-line drivers for running taint analysis, generating representative paths, previewing LLM prompts, and executing the staged LLM workflow.

The implementation is organized around three major packages. The `taint_analysis` package contains the static-analysis engine, source and sink definitions, path tracking, path reconstruction, source canonicalization, prompt generation, and Slither integration. The `llm_analysis` package contains the staged LLM engine and prompt templates. The dataset and script directories contain audit-finding collection, repository cloning, build verification, and dataset validation utilities.

4.2 Analyzer Orchestration

The main analyzer object is `PriceManipulationAnalyzer`. It receives a contract path and an optional taint-analysis configuration. It then initializes three core components:

doi:10.5566/NTU2026XXXXX

- `SlitherAnalyzer`, which parses the target project and exposes contracts, functions, CFG nodes, and SlithIR operations.
- `SourceSinkManager`, which manages taint source and sink definitions.
- `TaintTracker`, which performs source identification, propagation, sink matching, and path reconstruction.

The analyzer returns a dictionary containing the target path, total vulnerable path count, path summaries, serialized paths, number of sources found, and number of sinks found. Each serialized path includes source file and line, source contract and function, source type, canonical source ID, sink file and line, sink kind, sink operand selector, tainted variable, contract, and function.

4.3 Source and Sink Configuration

The implementation defines configurable patterns for common price sources and sinks. Oracle patterns include function names such as `latestRoundData`, `getPrice`, `getReserves`, `consult`, and `fetchPrice`. Flash-loan patterns include `flashLoan`, `borrow`, and `executeOperation`. Price calculation sinks include swap and quote helpers, while collateral and lending sinks include liquidation, health factor, borrowing, depositing, withdrawing, minting, burning, and repayment patterns.

The configuration also supports storage-taint sources based on price-like state variable names such as `price`, `rate`, `oracle`, and `feed`. During initialization, the analyzer scans state variables in the target contracts and constructs a storage-read source for matched variables.

doi:10.5566/NTU2026XXXXX

4.4 Taint Engine

The `TaintTracker` implements the core static analysis. It maintains a taint map, a path tracker, source records, raw source candidates, sink matches, return summaries, call contexts, and worklist metrics. The engine operates at the project level rather than only within a single contract. This is necessary because DeFi protocols often split price logic, accounting logic, and token operations across multiple contracts and libraries.

The engine identifies raw source candidates and converts them into canonical source records. A source record stores a canonical ID, display label, caller source label, source type, exact seed IDs, provenance chain, terminal external sites, source node, source operation, and related caller metadata. This design allows a source wrapper to remain connected to the underlying price read.

Propagation is delegated to operation-specific handlers. These handlers process assignments, binary operations, calls, type conversions, member accesses, tuple unpacking, and returns. The implementation also maintains return summaries so that a tainted return value can be propagated from a callee back to caller contexts. This is important for helper functions that wrap price reads.

4.5 Path Tracking and Grouping

The `PathTracker` stores vulnerable paths and source aliases. It maps exact variable identifiers to canonical source IDs, tracks reporting seed IDs, and records source realizations created during propagation. This separation between exact IDs and canonical IDs is important because Solidity compilation and Slither SSA can produce many local identi-

doi:10.5566/NTU2026XXXXX

fiers for one semantic source.

Raw vulnerable paths are grouped into representative paths before LLM analysis. The grouping layer reduces duplicate paths, merges sink-function records, and preserves marked sink lines. The resulting prompt entries contain source records, sink groups, sink function code, state writes, external calls, representative path counts, raw path counts, and source-sink chains.

This abstraction is a practical requirement. In representative runs, one project can produce hundreds of raw paths. Passing all of them directly to an LLM would increase cost and would also make the final report difficult to interpret.

4.6 Prompt Generation

The prompt generator renders concise evidence from representative paths. It normalizes whitespace, truncates long statements, formats source provenance, extracts line labels, and renders source-to-sink chains. It also marks sink lines in function bodies so that the LLM can inspect the relevant code without receiving unrelated files.

The generated prompt JSON is saved under a project-specific output directory. This makes each analysis run auditable: the final LLM report can be traced back to the exact prompt entries and static-analysis summary used for that run.

4.7 LLM Engine

The `LLMEngine` orchestrates four named stages: `classify_canonical_sources`, `classify_sources_with_context`, `triage_sink_group_impact`, and `judge_root_cause_findings`

doi:10.5566/NTU2026XXXXX

The engine supports preview mode, which prints prompts without calling an API, and run mode, which calls the configured OpenAI model and writes a structured JSON report.

Every stage validates the model response. Source classification must return one result for every expected source label and must use one of the allowed source classes and decisions. Sink triage must return one result for every expected sink group ID. Root-cause judgment must return one result for every expected canonical source ID and must refer only to known evidence IDs.

This validation does not prove that the model is correct, but it prevents many format and traceability failures. It also makes the report suitable for automatic matching against ground truth.

4.8 Dataset Utilities

The implementation includes scripts for collecting and organizing audit findings. The dataset CSV records report date, audit firm, severity, protocol, title, tags, year, compilation status, source-code link, Solodit URL, GitHub link, summary, manipulation label, source-code availability, and remarks. This format supports filtering from a broad collected set to a narrower build-verified evaluation set.

A separate build-manifest verifier checks curated HIGH-severity projects. It validates project IDs, roots, toolchains, build commands, compiler versions, required environment fields, lockfiles, and build output directories. This is important because a finding with a public repository is not automatically usable for static analysis. The project must be available in a reproducible form that Slither can analyze.

doi:10.5566/NTU2026XXXXX

4.9 Execution Modes

The main taint-analysis test driver can run in three LLM modes:

- `off`: run taint analysis and generate artifacts only.
- `preview`: render the staged LLM prompts without API calls.
- `run`: execute the configured LLM workflow and write a report.

The LLM engine also has a standalone test driver. Given an existing `llm_prompts` JSON file, it can preview prompts, run the workflow, or step through prompts interactively. This separation makes it possible to debug static analysis and LLM reasoning independently.

4.10 Implementation Limitations

The implementation delivered in this thesis focuses on taint analysis, representative path generation, and staged LLM report generation. Sanity-verification rules are still limited. In particular, checks for access control, TWAP adequacy, staleness validation, slippage protection, fee-on-transfer behavior, and time-based controls are not yet modeled in a systematic way.

The sink definitions also remain incomplete. The case studies in Chapter 5 show cases where canonical sources are found but no sink sites are retained. This can happen when the analyzed target does not include the relevant code path, or when the sink model does not cover the protocol's specific economic operation. These limitations are reported

doi:10.5566/NTU2026XXXXX

explicitly rather than hidden.

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

Chapter 5 Evaluation

5.1 Evaluation Goals

The evaluation is designed to answer whether the workflow produces reports that are useful for auditors. This differs from a pure detector evaluation. A detector evaluation asks whether the system predicts that a project is vulnerable. An audit-usefulness evaluation asks whether the system points to the right code, explains the right root cause, and avoids excessive false reports.

The evaluation therefore has three goals:

1. Measure protocol-level detection performance for compatibility with prior work.
2. Measure report-level correctness, including localization and explanation quality.
3. Measure over-reporting, because unsupported extra findings increase manual audit cost.

5.2 Dataset

The evaluation dataset is built from public audit findings. Each record contains meta-data such as report date, audit firm, severity, protocol, title, tags, year, source-code link,

doi:10.5566/NTU2026XXXXX

Solodit URL, GitHub link, summary, whether the issue is classified as price, reward, or vote manipulation, whether source code is available, and remarks.

The dataset is organized into three subsets.

Collected findings include all candidate findings from the collection pipeline. This set is useful for understanding the distribution of protocols, firms, severities, and issue categories, but it may contain entries that are not directly analyzable.

Source-available findings include records with public source code. This set is closer to the target evaluation but still may include repositories that are difficult to build or incomplete.

Build-verified findings include records whose repositories can be compiled or analyzed reproducibly. This is the primary subset for quantitative evaluation because the static-analysis front end depends on buildable Solidity projects.

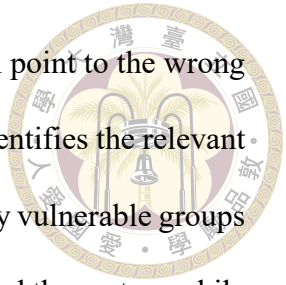
This separation avoids overstating the dataset size. A large collected dataset does not automatically imply a large reproducible evaluation set.

5.3 Metrics

5.3.1 Why Binary Metrics Are Insufficient

Precision, recall, and F1-score are standard metrics for vulnerability detection. They are easy to compute and useful for comparing classifiers. For audit assistance, however, they are incomplete. A binary true positive can hide multiple report-level failures that matter in practice.

doi:10.5566/NTU2026XXXXX



First, a system may correctly flag a project as vulnerable but still point to the wrong code path. For an auditor, this is much less useful than a report that identifies the relevant source, sink, or affected function. Second, a system may emit too many vulnerable groups or findings. If only one report is correct, binary scoring may still reward the system while ignoring the extra manual review cost of unsupported reports. Third, a system may localize roughly the right region but explain the wrong root cause or recommend the wrong mitigation. In that case, the output still requires substantial human reinterpretation before it becomes audit-useful.

These limitations motivate report-level evaluation alongside protocol-level metrics. In this thesis, a generated report is therefore additionally judged by localization accuracy, explanation quality, mitigation agreement, semantic similarity to the corresponding audit finding, and the number of unsupported extra reports. Binary metrics remain useful as a baseline, but they are not treated as the final measure of success.

5.3.2 Binary Metrics

The first metric group follows standard vulnerability detection evaluation. A project-level prediction is positive if the system emits at least one price-manipulation finding for the project. Ground truth is positive if the project contains at least one relevant audit finding. From these labels, the evaluation computes true positives, false positives, false negatives, precision, recall, and F1-score.

These metrics are reported mainly for compatibility with prior work such as PMDetector [5]. Their role in this thesis is to provide a baseline view of detector behavior before moving to more detailed report-level measurements.

doi:10.5566/NTU2026XXXXX

5.3.3 Localization Metrics

The second metric group evaluates whether a generated report points to the right place. Localization can be measured at several levels:

- Protocol-level localization: the prediction belongs to the same project as the ground-truth finding.
- Contract or function localization: the predicted root-cause location names the same contract, function, wrapper, or semantically equivalent code region.
- Source localization: the predicted vulnerable read corresponds to the same price source or source wrapper.
- Sink localization: the predicted sink corresponds to the same economic operation as the audit finding.
- Mitigation localization: the predicted missing mitigation matches the ground-truth mitigation category.

These levels make partial success visible. For example, a prediction may identify the correct spot price source but miss the exact liquidation sink. Another prediction may identify the affected function but give the wrong mitigation.

5.3.4 Semantic Explanation Score

The explanation score compares the generated explanation with the audit-finding summary. The evaluation embeds both texts using a sentence embedding model and computes cosine similarity:

doi:10.5566/NTU2026XXXXX

$$\text{DescriptionScore}(p, g) = \begin{cases} \cos(E(p), E(g)), & \cos(E(p), E(g)) > \tau \\ 0, & \text{otherwise} \end{cases}$$

where p is the predicted explanation, g is the ground-truth explanation, $E(\cdot)$ is the embedding function, and τ is the similarity threshold. In this thesis, the threshold is set to 0.7, inspired by semantic evaluation settings such as the Bastet competition [7]. This fixed threshold provides a consistent operating point for report matching, although its sensitivity remains a validity consideration discussed later in this chapter.

5.3.5 Over-Reporting Penalty

The over-reporting penalty measures how many extra reports the system emits for a project:

$$\text{Penalty}(p) = \max(0, \#\text{Predictions}(p) - \#\text{GroundTruth}(p)).$$

This penalty is important because the static-analysis phase intentionally over-approximates candidate paths. Without a penalty, the system could improve recall by emitting many weak findings. In an audit setting, such behavior is costly because each report must be checked manually.

doi:10.5566/NTU2026XXXXX

5.4 Matching Procedure

For each project, the evaluation builds a pairwise score matrix between predicted reports and ground-truth audit findings. Each pair score combines localization agreement, mitigation agreement, and semantic explanation similarity. Scores below a minimum threshold are set to zero. Maximum-weight bipartite matching is then used to assign predictions to ground-truth findings.

After matching, each prediction is classified as matched or unmatched, and each ground-truth finding is classified as covered or missed. The evaluation reports the total matched score, unmatched predictions, missed findings, binary metrics, and over-reporting penalty.

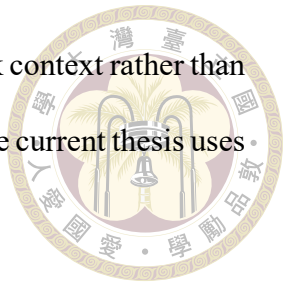
5.5 Baselines

The main baseline is a PMDetector-style binary evaluation of the current workflow. Under this baseline, a project is counted as correctly detected if at least one relevant vulnerability is found. This baseline intentionally ignores whether the report localizes the right path or explains the right root cause.

The evaluation framework also includes a simpler PMDetector-style prompt baseline that emits one report containing `is_high_risk`, vulnerable functions, vulnerable groups, and a reason. Comparing this baseline with the staged workflow would show whether decomposing source classification, sink triage, and root-cause judgment improves audit-useful output.

doi:10.5566/NTU2026XXXXX

The published PMDetector results should be treated as prior-work context rather than a direct baseline unless the dataset and implementation are aligned. The current thesis uses different ground truth and different evaluation metrics.



5.6 Qualitative Results

The implemented workflow produces analyzable artifacts for several projects. This section presents representative qualitative results that illustrate both the strengths and the limitations of the workflow.

5.6.1 Spartan

For the Spartan project, one analysis summary reports 25 analyzed contracts, 67 raw source candidates, 17 canonical sources, 33 sink sites, 217 raw vulnerable paths, and 97 representative paths. The LLM report filters eight source labels down to two retained spot-price sources: `iPOOL.baseAmount` and `iPOOL.tokenAmount`. It then marks 13 sink groups as dangerous and emits two vulnerable root-cause findings.

This result illustrates the intended pipeline behavior. Static analysis generates many raw paths, representative grouping reduces redundancy, and the LLM collapses evidence into a smaller number of root-cause findings. The key evaluation question is whether those findings align with the corresponding audit or incident ground truth.

doi:10.5566/NTU2026XXXXX

5.6.2 Predy

For Predy, the LLM report begins from 41 representative paths and six source labels. It keeps `IUniswapV3PoolState.slot0` and `Trade.getSqrtPrice`, drops fee, liquidity, TWAP, and protocol-state sources, and emits two vulnerable root-cause findings. The generated explanations point to spot-price use in `Perp.reallocate` and `Trade.trade`, with missing TWAP or slippage checks.

This result shows the value of source classification. Without the first LLM stage, liquidity and protocol-state values could become false positives. The stage decomposition helps focus root-cause judgment on price-like sources.

5.6.3 Abracadabra Money

For one Abracadabra Money run, the summary reports 87 analyzed contracts, 56 raw source candidates, and 23 canonical sources, but zero sink sites and zero representative paths. This is a useful failure case. It may indicate that the current sink definitions do not cover the relevant economic operation, that the selected analysis target does not include the affected code path, or that the issue requires modeling that is not yet implemented.

This case supports the need for transparent reporting. A binary evaluation would count the project as a miss if ground truth contains a relevant finding, but the engineering diagnosis is more specific: source identification may work, while sink coverage fails.

doi:10.5566/NTU2026XXXXX

5.7 Threats to Validity

Construct validity. Semantic similarity may not perfectly represent auditor judgment. The embedding model and threshold therefore approximate, but do not fully replace, human assessment.

Internal validity. LLM results may depend on model version, prompt wording, temperature, and chunking. These dependencies should be considered when interpreting the reported results.

External validity. The thesis focuses on price manipulation in Solidity DeFi protocols. Results may not generalize to other vulnerability classes, non-EVM chains, or protocols with off-chain components.

Dataset validity. Audit findings differ in specificity and quality. Some findings are disputed, duplicated, or only partially related to price manipulation. The dataset therefore requires explicit inclusion and exclusion criteria.

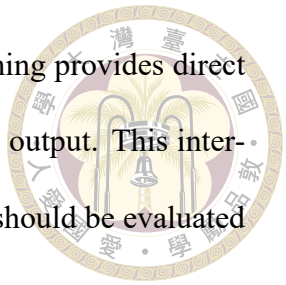
Implementation validity. The current implementation may miss paths because of dynamic dispatch, proxy patterns, incomplete build targets, missing sink definitions, or unsupported Solidity patterns. These failures are reported explicitly rather than hidden.

5.8 Evaluation Interpretation

The main contribution of this evaluation design is a clearer measurement of audit usefulness. A system may have good protocol-level detection while still showing poor localization, weak explanation matching, or high over-reporting; in such cases, binary

doi:10.5566/NTU2026XXXXX

metrics alone are misleading. Conversely, stronger report-level matching provides direct evidence that staged source and sink reasoning improves audit-useful output. This interpretation supports the central thesis that LLM-assisted security tools should be evaluated at the report level rather than only through binary labels.



doi:10.5566/NTU2026XXXXX

Chapter 6 Conclusion and Future Work

6.1 Conclusion

This thesis studies how PMDetector-style workflows can be evaluated as audit-assistance systems rather than only as binary vulnerability classifiers. Price manipulation vulnerabilities are difficult to detect because the dangerous behavior depends on both data flow and economic context. A price-like source is not always vulnerable, and an economically meaningful sink is not always controlled by a manipulable value. Static analysis can identify possible flows, but it tends to over-approximate. LLMs can reason about code context and missing mitigations, but they require concrete evidence and structured output to be useful.

The system developed for this thesis combines these two strengths. It uses Slither-based taint analysis to identify candidate flows, canonical source records to merge equivalent sources, representative path generation to reduce raw path redundancy, and a staged LLM workflow to classify sources, triage sink impact, and emit audit-style root-cause findings. The output is designed to be traceable: each final finding refers back to concrete static-analysis evidence.

doi:10.5566/NTU2026XXXXX

The main argument of this thesis is that protocol-level precision, recall, and F1-score are insufficient for evaluating such systems. A detector can correctly label a project as vulnerable while still pointing to the wrong path, naming the wrong function, explaining the wrong mitigation, or producing many extra reports. These failures matter because auditors do not only need a yes-or-no answer. They need precise, actionable, and low-noise reports.

To address this gap, the thesis proposes a report-level evaluation framework based on audit findings. The framework combines binary metrics with localization checks, semantic explanation similarity, mitigation agreement, and an over-reporting penalty. This design is intended to measure whether generated reports are actually useful in an audit workflow.

6.2 Limitations

The implementation delivered in this thesis is a working system with components for taint analysis, source and sink tracking, path grouping, LLM prompt generation, staged LLM execution, JSON validation, and dataset organization. The qualitative results in Chapter 5 show that the workflow can produce plausible findings for projects such as Spartan and Predy. They also expose important limitations, especially incomplete sink coverage in some projects.

The evaluation scope of this thesis is therefore centered on framework design, report structure, and case-based analysis rather than on a fully frozen large-scale benchmark. Although the collected audit-finding CSV and the HIGH dataset slice provide useful metadata and build-readiness checks, exhaustive manual labeling and large-scale quantitative

doi:10.5566/NTU2026XXXXX

benchmarking remain outside the scope of the present thesis.

6.3 Future Work

6.3.1 Complete Sanity Verification

The most important implementation task is to complete sanity verification. Static reachability and LLM reasoning should be checked against concrete mitigation patterns. Future rules should cover TWAP adequacy, oracle staleness, deviation checks, slippage checks, access control, time-based controls, fee-on-transfer behavior, and protocol-specific guard conditions. These checks would reduce false positives and make LLM findings more defensible.

6.3.2 Expand Sink Coverage

The case studies in Chapter 5 show that source identification can succeed while sink matching fails. Future work should expand sink definitions beyond the current patterns. In particular, the analyzer should better capture protocol-specific accounting updates, vault share calculations, liquidity-position updates, reward eligibility checks, and liquidation-adjacent helper functions. Each new sink definition should be tested against real audit findings.

6.3.3 Freeze a Build-Verified Dataset

Future large-scale evaluations should use a frozen dataset split. Each project should record the repository commit, compiler version, toolchain, build command, required en-

doi:10.5566/NTU2026XXXXX

vironment variables, and analysis target. Findings should be categorized as price manipulation, related manipulation, or out of scope. Without a frozen dataset, results will be hard to reproduce and compare.

6.3.4 Calibrate Semantic Matching

The semantic explanation score in this thesis uses a fixed threshold of 0.7. Future work should build a small validation set of prediction-ground-truth pairs labeled by human reviewers. The embedding model and threshold should then be calibrated against this validation set. This step is necessary before semantic similarity can be treated as a reliable metric.

6.3.5 Implement Stronger Baselines

A direct PMDetector reproduction is difficult without the original source code, but useful baselines are still possible. One baseline can use the same static analysis front end with a single PMDetector-style prompt that emits `is_high_risk`, vulnerable functions, vulnerable groups, and a reason. Another baseline can evaluate static analysis alone without LLM filtering. Comparing these baselines with the staged workflow would clarify which parts of the design improve report-level usefulness.

6.3.6 Human Auditor Study

The strongest validation would involve human auditors. Given a set of generated reports, auditors could rate whether each report helps them locate the issue, understand the root cause, and decide on mitigation. Such a study would directly evaluate the thesis

doi:10.5566/NTU2026XXXXX

claim that report-level metrics better capture audit usefulness than binary labels alone.

6.4 Final Remarks

LLM-assisted security tools should not be evaluated only by whether they can say that a project is vulnerable. For practical auditing, the harder and more important question is whether they can explain the vulnerability accurately, localize it precisely, and avoid unnecessary noise. This thesis takes a step toward that goal by implementing a PMDetector-inspired workflow and proposing an audit-centered evaluation framework for price manipulation detection.

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

References

[1] J. Chen, Y. Shen, J. Zhang, Z. Li, J. Grundy, Z. Shao, Y. Wang, J. Wang, T. Chen, and Z. Zheng. FORGE: An LLM-driven framework for large-scale smart contract vulnerability dataset construction. arXiv:2506.18795, 2025. Accepted by ICSE 2026.

[2] Code4rena. Code4rena audits. https://code4rena.com/audits, 2026. Accessed 2026-07-09.

[3] Cyfrin. Solodit. https://solodit.cyfrin.io/, 2026. Accessed 2026-07-09.

[4] S. Hu, T. Huang, F. İlhan, S. F. Tekin, and L. Liu. Large language model-powered smart contract vulnerability detection: New perspectives. arXiv:2310.01152, 2023. Introduces the GPTLens framework.

[5] L. Liu, W. Zhang, L. Wei, H. Guan, Y. Tian, Y. Liu, and S.-C. Cheung. LLM-powered detection of price manipulation in DeFi. arXiv:2510.21272v2, 2026. Last revised 29 Apr 2026.

[6] B. Mueller. Hound: Relation-first knowledge graphs for complex-system reasoning in security audits. arXiv:2510.09633, 2025.

[7] OneSavie. Onesavie bastet competition. https://www.kaggle.com/competitions/onesavie-bastet/overview, 2026. Accessed 2026-07-09.

doi:10.5566/NTU2026XXXXX

[8] OpenAI. Introducing EVMbench. https://openai.com/index/introducing-evmbench/, 2026. Accessed 2026-07-17.

[9] ScaBench. ScaBench: Smart contract ai benchmark. https://github.com/scabench-org/scabench, 2026. Accessed 2026-07-17.

[10] Sherlock. Sherlock audit contests. https://audits.sherlock.xyz/, 2026. Accessed 2026-07-09.

[11] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, and Y. Liu. GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis. arXiv:2308.03314, 2024. Accepted by ICSE 2024.

[12] Trail of Bits. Slither: Static analyzer for Solidity. https://github.com/crytic/slither, 2026. Accessed 2026-07-09.

[13] Z. Wei, J. Sun, Z. Zhang, Z. Hou, and Z. Zhao. Adaptive plan-execute framework for smart contract security auditing. arXiv:2505.15242, 2025. SmartAuditFlow.

[14] Z. Wei, J. Sun, Z. Zhang, X. Zhang, M. Li, and Z. Hou. LLM-SmartAudit: Advanced smart contract vulnerability detection. arXiv:2410.09381, 2024.

[15] H. Wu, H. Wang, S. Li, Y. Wu, M. Fan, Y. Zhao, and T. Liu. Detecting state manipulation vulnerabilities in smart contracts using LLM and static analysis. arXiv:2506.08561v2, 2025.

[16] S. Wu, D. Wang, J. He, Y. Zhou, L. Wu, X. Yuan, Q. He, and K. Ren. DeFiRanger: Detecting price manipulation attacks on DeFi applications. arXiv:2104.15068, 2021.

[17] L. Yu, S. Chen, H. Yuan, P. Wang, Z. Huang, J. Zhang, C. Shen, F. Zhang, L. Yang,

doi:10.5566/NTU2026XXXXX

and J. Ma. Smart-LLaMA: Two-stage post-training of large language models for smart contract vulnerability detection and explanation. arXiv:2411.06221, 2024.

[18] X. Zhang, Z. Gao, Y. Lv, X. Hu, F. Niu, and X. Xia. On the shoulders of giants: Empowering automated smart contract auditing via the GiAnt corpus. arXiv:2606.07363, 2026.

[19] J. Zhong, D. Wu, Y. Liu, M. Xie, Y. Liu, Y. Li, and N. Liu. Detecting various DeFi price manipulations with LLM reasoning. arXiv:2502.11521v2, 2025. Accepted by ASE 2025.

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

Appendix A — Introduction

A.1 Introduction

A.2 Further Introduction

doi:10.5566/NTU2026XXXXX

doi:10.5566/NTU2026XXXXX

Appendix B — Introduction

B.1 Introduction

B.2 Further Introduction

doi:10.5566/NTU2026XXXXX